

Boolean algebra

The logical symbol 0 and 1 are used for representing the digital input or output. The symbols "1" and "0" can also be used for a permanently open and closed digital circuit. The digital circuit can be made up of several logic gates. To perform the logical operation with minimum logic gates, a set of rules were invented, known as the Laws of Boolean Algebra. These rules are used to reduce the number of logic gates for performing logic operations.

The Boolean algebra is mainly used for simplifying and analysing the complex Boolean expression. It is also known as **Binary algebra** because we only use binary numbers in this. **George Boole** developed the binary algebra in **1854**.

Rules in Boolean algebra

1. Only two values (1 for high and 0 for low) are possible for the variable used in Boolean algebra.
2. The over bar (-) is used for representing the complement variable. So, the complement of variable C is represented as $\bar{C}$.
3. The plus (+) operator is used to represent the OR of the variables.
4. The dot (.) operator is used to represent the AND of the variables.

Properties of Boolean algebra

These are the following properties of Boolean algebra:

Annulment Law

When the variable is AND with 0, it will give the result 0, and when the variable is OR with 1, it will give the result 1, i.e.,

$$B \cdot 0 = 0$$

$$B + 1 = 1$$

Identity Law

When the variable is AND with 1 and OR with 0, the variable remains the same, i.e.,

$$B \cdot 1 = B$$

$$B+0 = B$$

Idempotent Law

When the variable is AND and OR with itself, the variable remains same or unchanged, i.e.,

$$B.B = B$$

$$B+B = B$$

Complement Law

When the variable is AND and OR with its complement, it will give the result 0 and 1 respectively.

$$B.B' = 0$$

$$B+B' = 1$$

Double Negation Law

This law states that, when the variable comes with two negations, the symbol gets removed and the original variable is obtained.

$$((A)')' = A$$

Commutative Law

This law states that no matter in which order we use the variables. It means that the order of variables doesn't matter in this law.

$$A.B = B.A$$

$$A+B = B+A$$

Associative Law

This law states that the operation can be performed in any order when the variables priority is of same as '*' and '/'.
priority is of same as '*' and '/'.

$$(A.B).C = A.(B.C)$$

$$(A+B)+C = A+(B+C)$$

Distributive Law

This law allows us to open up of brackets. Simply, we can open the brackets in the Boolean expressions.

$$A+(B.C) = (A+B).(A+C)$$

$$A.(B+C) = (A.B)+(A.C)$$

Absorption Law

This law allows us for absorbing the similar variables.

$$B+(B.A) = B$$

$$B.(B+A) = B$$

De Morgan Law

The operation of an OR and AND logic circuit will remain same if we invert all the inputs, change operators from AND to OR and OR to AND, and invert the output.

$$(A.B)' = A'+B'$$

$$(A+B)' = A'.B'$$

Laws and Rules of Boolean algebra

In simplification of the Boolean expression, the laws and rules of the Boolean algebra play an important role. Before understanding these laws and rules of Boolean algebra, understand the Boolean operations addition and multiplication concept.

Boolean Addition

The addition operation of Boolean algebra is similar to the OR operation. In digital circuits, the OR operation is used to calculate the sum term, without using AND operation. $A + B$, $A + B'$, $A + B + C'$, and $A' + B + + D'$ are some of the examples of 'sum term'. The value of the sum term is true when one or more than one literals are true and false when all the literals are false.

Boolean Multiplication

The multiplication operation of Boolean algebra is similar to the AND operation. In digital circuits, the AND operation calculates the product, without using OR

operation. AB, AB, ABC, and ABCD are some of the examples of the product term. The value of the product term is true when all the literals are true and false when any one of the literal is false.

Laws of Boolean algebra

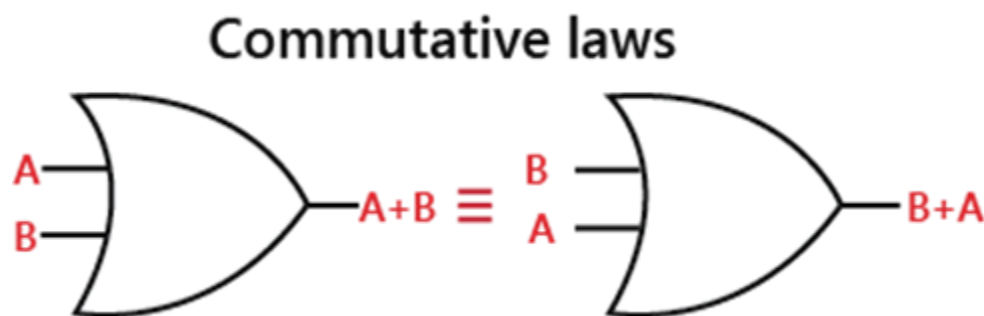
There are the following laws of Boolean algebra:

Commutative Law

This law states that no matter in which order we use the variables. It means that the order of variables doesn't matter. In Boolean algebra, the OR and the addition operations are similar. In the below diagram, the OR gate display that the order of the input variables does not matter at all.

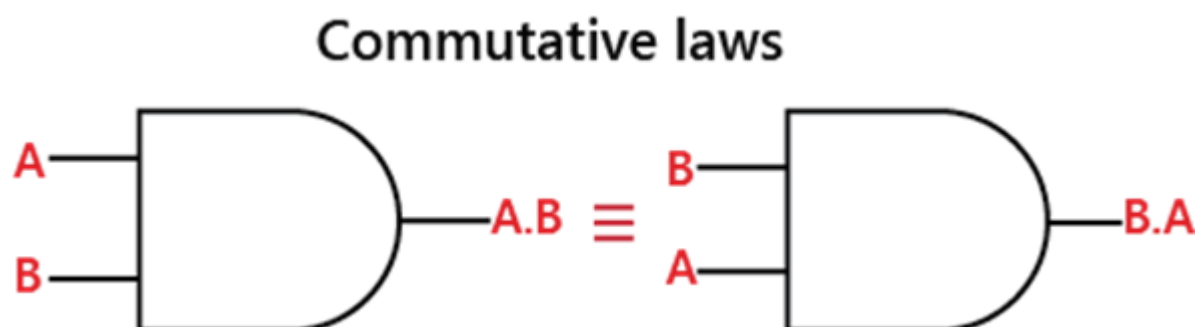
For two variables, the commutative law of addition is written as:

$$A+B = B+A$$



For two variables, the commutative law of multiplication is written as:

$$A.B = B.A$$

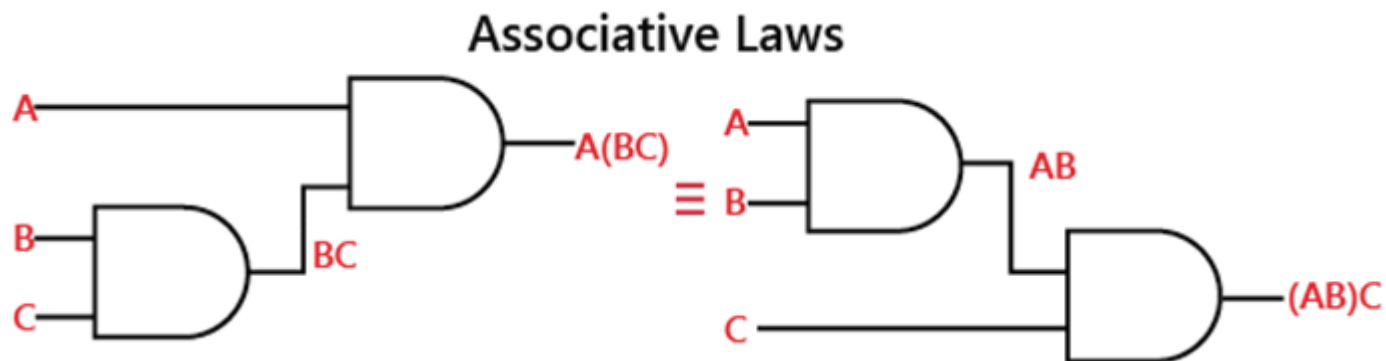


Associative Law

This law states that the operation can be performed in any order when the variables priority is same. As '*' and '/' have same priority. In the below diagram, the associative law is applied to the 2-input OR gate.

For three variables, the associative law of addition is written as:

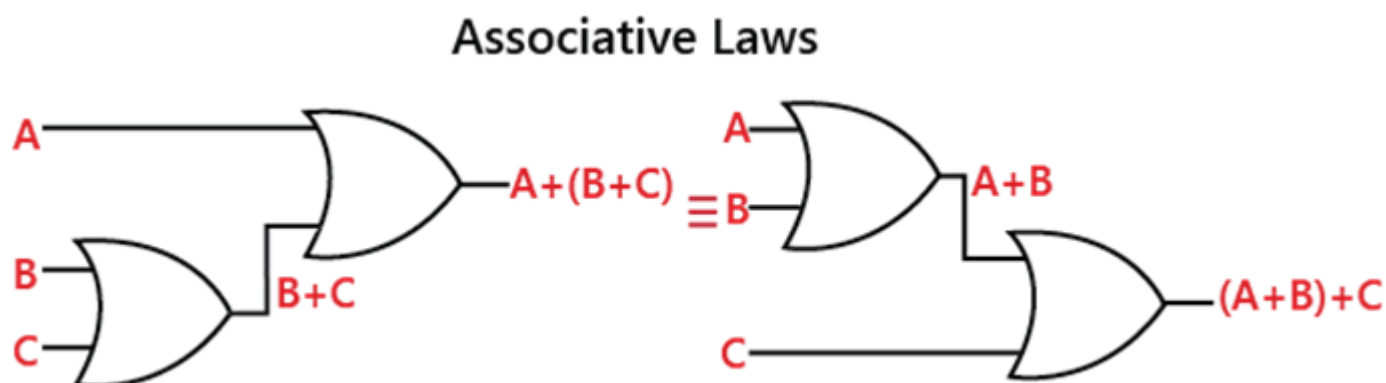
$$A + (B + C) = (A + B) + C$$



For three variables, the associative law of multiplication is written as:

$$A(BC) = (AB)C$$

According to this law, no matter in what order the variables are grouped when ANDing more than two variables. In the below diagram, the associative law is applied to 2-input AND gate.

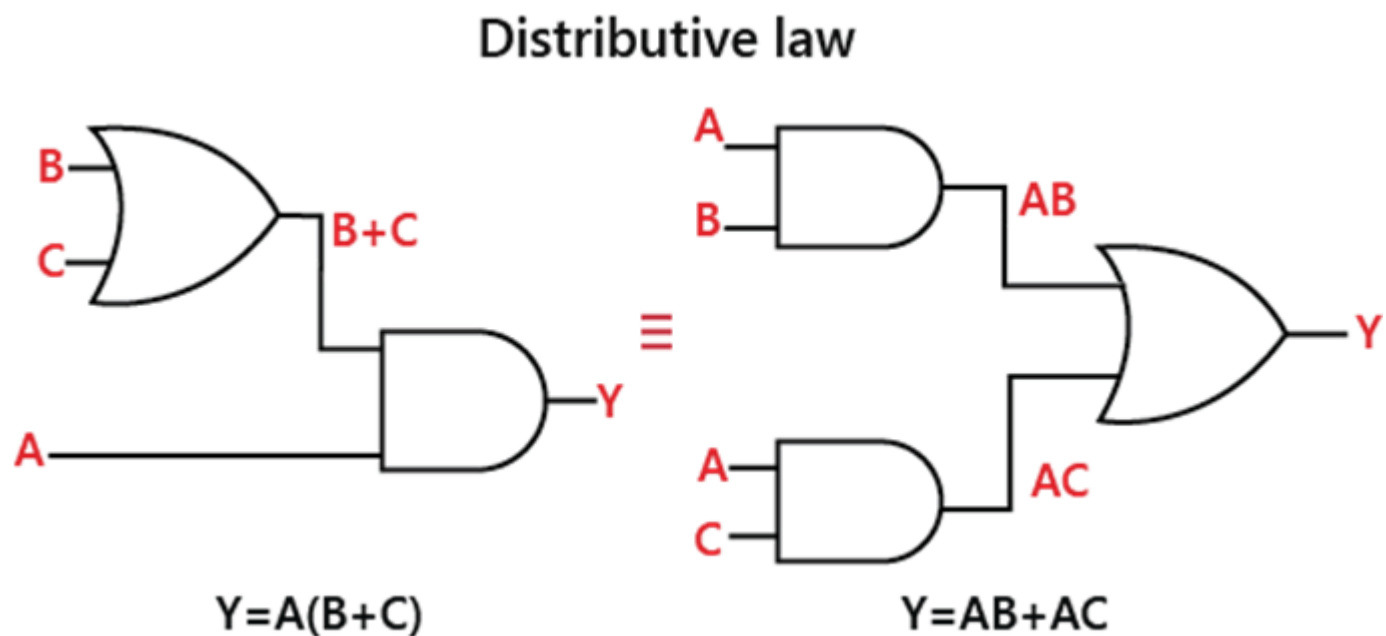


Distributive Law:

According to this law, if we perform the OR operation of two or more variables and then perform the AND operation of the result with a single variable, then the result will be similar to performing the AND operation of that single variable with each two or more variable and then perform the OR operation of that product. This law explains the process of factoring.

For three variables, the distributive law is written as:

$$A(B + C) = AB + AC$$



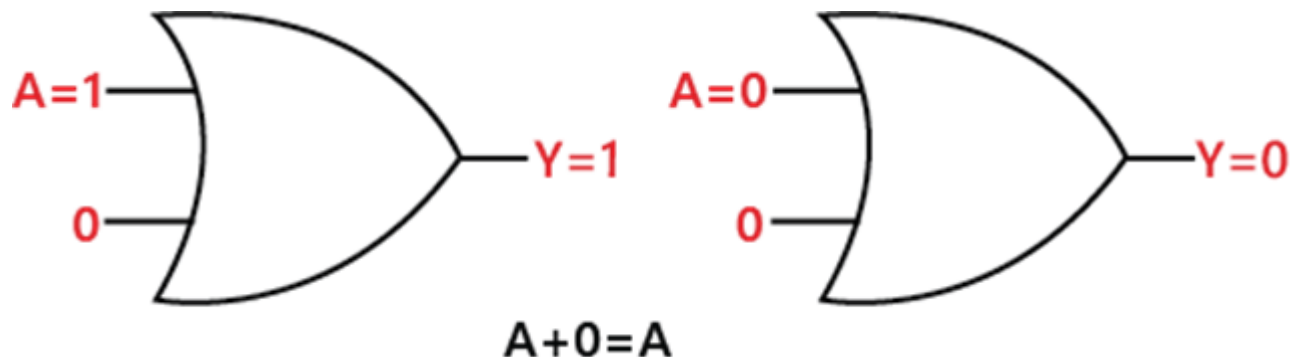
Rules of Boolean algebra

There are the following rules of Boolean algebra, which are mostly used in manipulating and simplifying Boolean expressions. These rules play an important role in simplifying boolean expressions.

1.	$A + 0 = A$	7.	$A.A = A$
2.	$A + 1 = 1$	8.	$A.A' = 0$
3.	$A.0 = 0$	9.	$A'' = A$
4.	$A.1 = A$	10.	$A + AB = A$
5.	$A + A = A$	11.	$A + A'B = A + B$
6.	$A + A' = 1$	12.	$(A + B)(A + C) = A + BC$

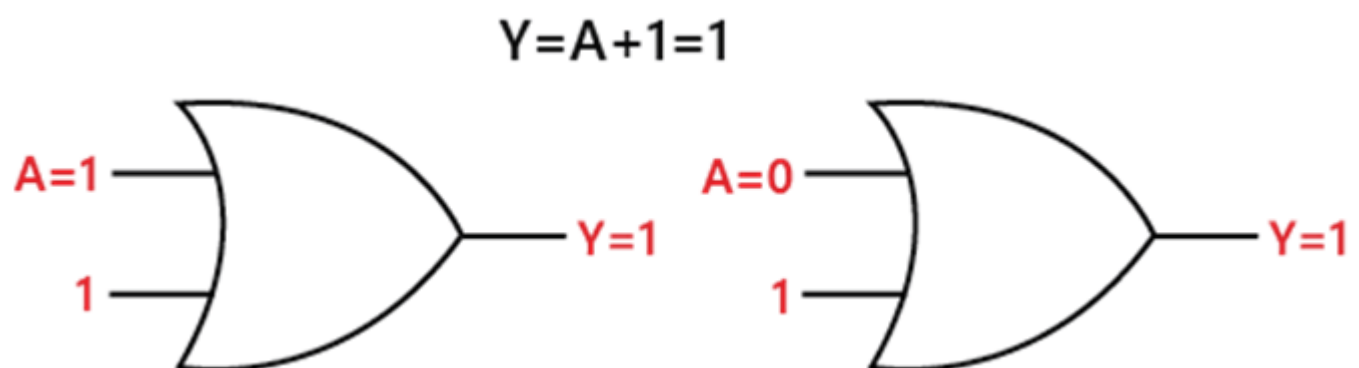
Rule 1: $A + 0 = A$

Let's suppose; we have an input variable A whose value is either 0 or 1. When we perform OR operation with 0, the result will be the same as the input variable. So, if the variable value is 1, then the result will be 1, and if the variable value is 0, then the result will be 0. Diagrammatically, this rule can be defined as:



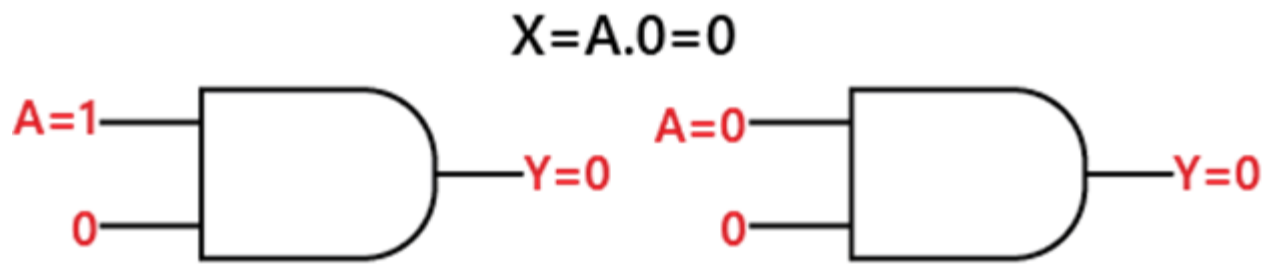
Rule 2: $(A + 1) = 1$

Let's suppose; we have an input variable A whose value is either 0 or 1. When we perform OR operation with 1, the result will always be 1. So, if the variable value is either 1 or 0, then the result will always be 1. Diagrammatically, this rule can be defined as:



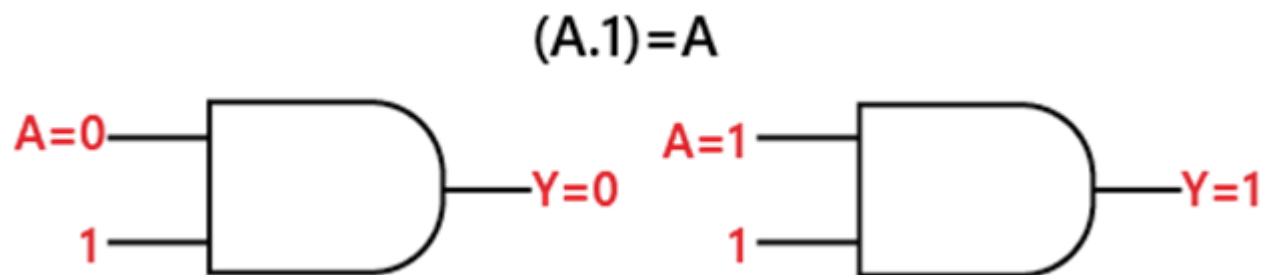
Rule 3: $(A.0) = 0$

Let's suppose; we have an input variable A whose value is either 0 or 1. When we perform the AND operation with 0, the result will always be 0. This rule states that an input variable ANDed with 0 is equal to 0 always. Diagrammatically, this rule can be defined as:



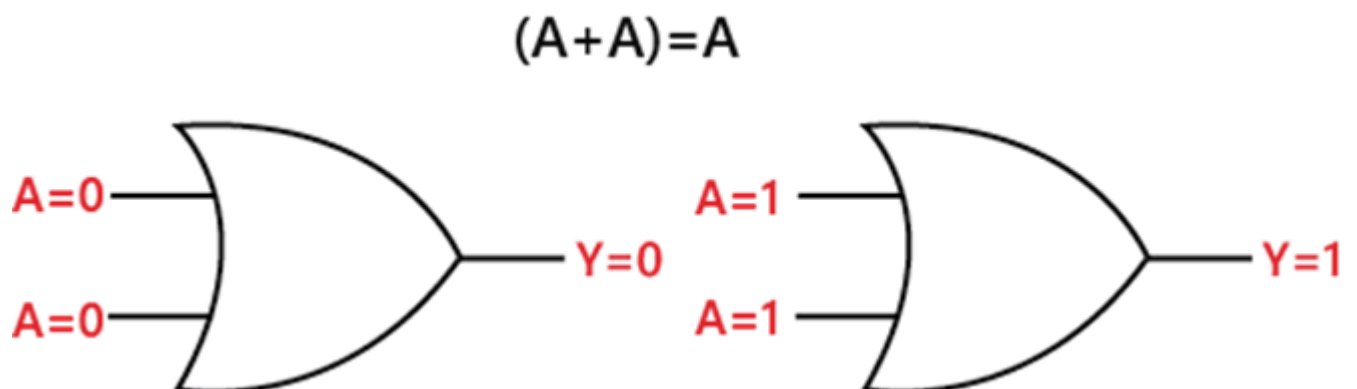
Rule 4: $(A \cdot 1) = A$

Let's suppose; we have an input variable A whose value is either 0 or 1. When we perform the AND operation with 1, the result will always be equal to the input variable. This rule states that an input variable ANDed with 1 is equal to the input variable always. Diagrammatically, this rule can be defined as:



Rule 5: $(A + A) = A$

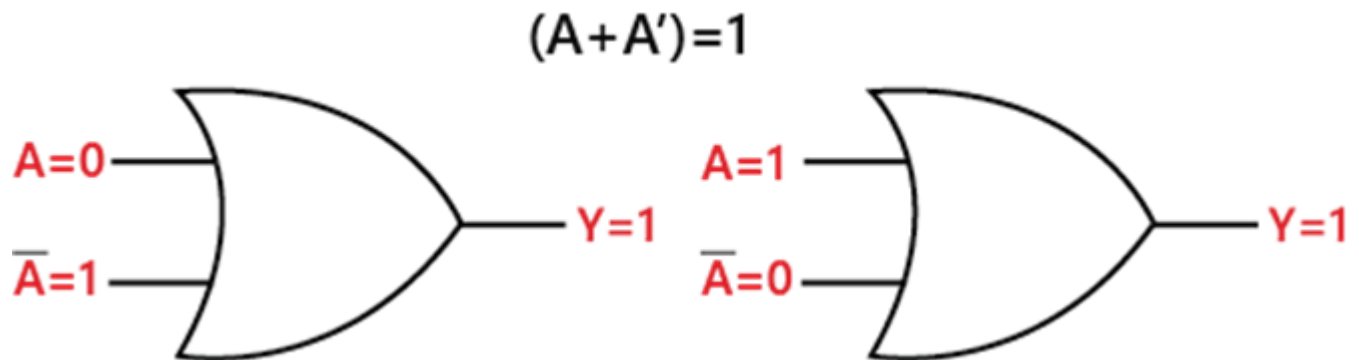
Let's suppose; we have an input variable A whose value is either 0 or 1. When we perform the OR operation with the same variable, the result will always be equal to the input variable. This rule states an input variable ORed with itself is equal to the input variable always. Diagrammatically, this rule can be defined as:



Rule 6: $(A + A') = 1$

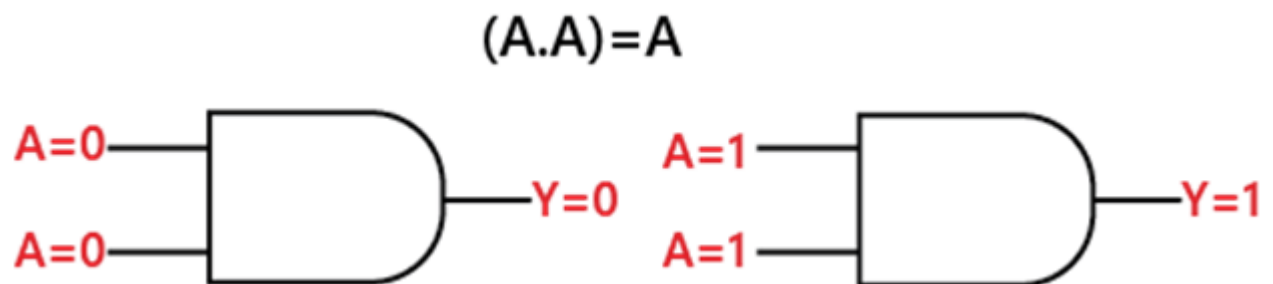
Let's suppose; we have an input variable A whose value is either 0 or 1. When we perform the OR operation with the complement of that variable, the result will always

be equal to 1. This rule states that a variable ORed with its complement is equal to 1 always. Diagrammatically, this rule can be defined as:



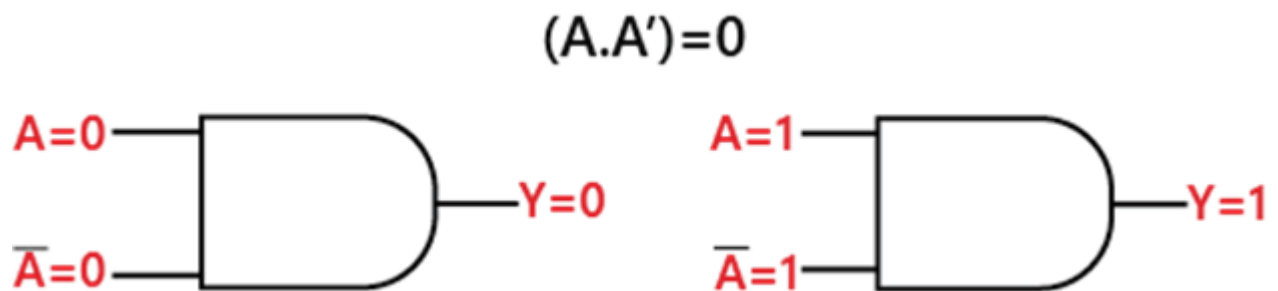
Rule 7: $(A.A) = A$

Let's suppose; we have an input variable A whose value is either 0 or 1. When we perform the AND operation with the same variable, the result will always be equal to that variable only. This rule states that a variable ANDed with itself is equal to the input variable always. Diagrammatically, this rule can be defined as:



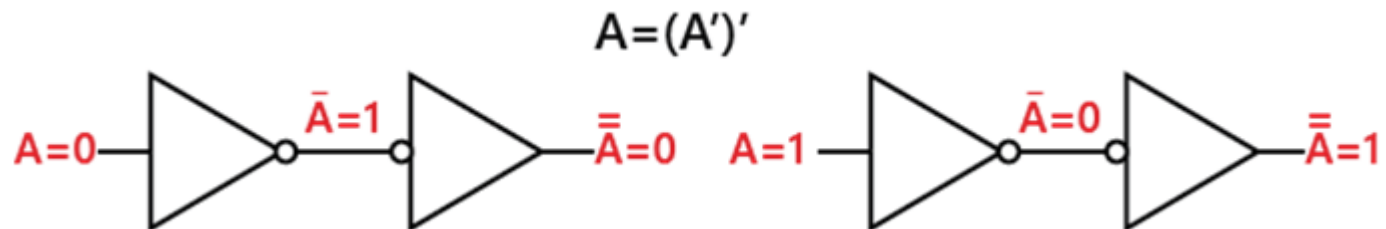
Rule 8: $(A.A') = 0$

Let's suppose; we have an input variable A whose value is either 0 or 1. When we perform the AND operation with the complement of that variable, the result will always be equal to 0. This rule states that a variable ANDed with its complement is equal to 0 always. Diagrammatically, this rule can be defined as:



Rule 9: $A = (A')'$

This rule states that if we perform the double complement of the variable, the result will be the same as the original variable. So, when we perform the complement of variable A, then the result will be A'. Further if we again perform the complement of A', we will get A, that is the original variable.



Rule 10: $(A + AB) = A$

We can prove this rule by using the rule 2, rule 4, and the distributive law as:

$A + AB = A(1 + B)$ $A + AB = A.1$ $A + AB = A$	Factoring (distributive law) Rule 2: $(1 + B) = 1$ Rule 4: $A.1 = A$
---	--

A	B	AB	A+AB
0	0	0	0
0	1	0	0
1	0	0	1
1	1	1	1

Equal

Rule 11: $A + AB = A + B$

We can prove this rule by using the above rules as:

$A + AB = (A + AB) + AB$ $A + AB = (AA + AB) + AB$ $A + AB = AA + AB + AA + AB$ $A + AB = (A + A)(A + B)$ $A + AB = 1.(A + B)$ $A + AB = A + B$	Rule 10: $A = A + AB$ Rule 7: $A = AA$ Rule 8: adding $AA = 0$ Factoring Rule 6: $A + A = 1$
---	--

Rule 4: drop the 1

A	B	AB	A+AB	A+B
0	0	0	0	0
0	1	1	1	1
1	0	0	1	1
1	1	0	1	1

↑ equal ↑

Rule 12: $(A + B)(A + C) = A + BC$

We can prove this rule by using the above rules as:

$$(A + B)(A + C) = AA + AC + AB + BC$$

Distributive law

$$(A + B)(A + C) = A + AC + AB + BC$$

Rule 7: $AA = A$

$$(A + B)(A + C) = A(1 + C) + AB + BC$$

Rule 2: $1 + C = 1$

$$(A + B)(A + C) = A.1 + AB + BC$$

Factoring (distributive law)

$$(A + B)(A + C) = A(1 + B) + BC$$

Rule 2: $1 + B = 1$

$$(A + B)(A + C) = A.1 + BC$$

Rule 4: $A . 1 = A$

$$(A + B)(A + C) = A + BC$$

A	B	C	A+B	A+C	(A+B)(A+C)	BC	A+BC
0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	1	0	1	1	0	0	0
0	1	1	1	1	1	1	1
1	0	0	1	1	1	0	1
1	0	1	1	1	1	0	1
1	1	0	1	1	1	0	1
1	1	1	1	1	1	1	1

↑ equal ↑